

BrickJewels — AI-Native Growth Concierge

An end-to-end Databricks build for D2C fine-jewelry retail

Field Engineering AI Build · Anuj Lathi · August 2026 · 100% synthetic data

1. Business Context & The Problem It Solves

BrickJewels is a direct-to-consumer (D2C) fine-jewelry retailer. Like every modern jewelry brand, it faces four intertwined challenges that traditional analytics stacks solve only in disconnected pieces:

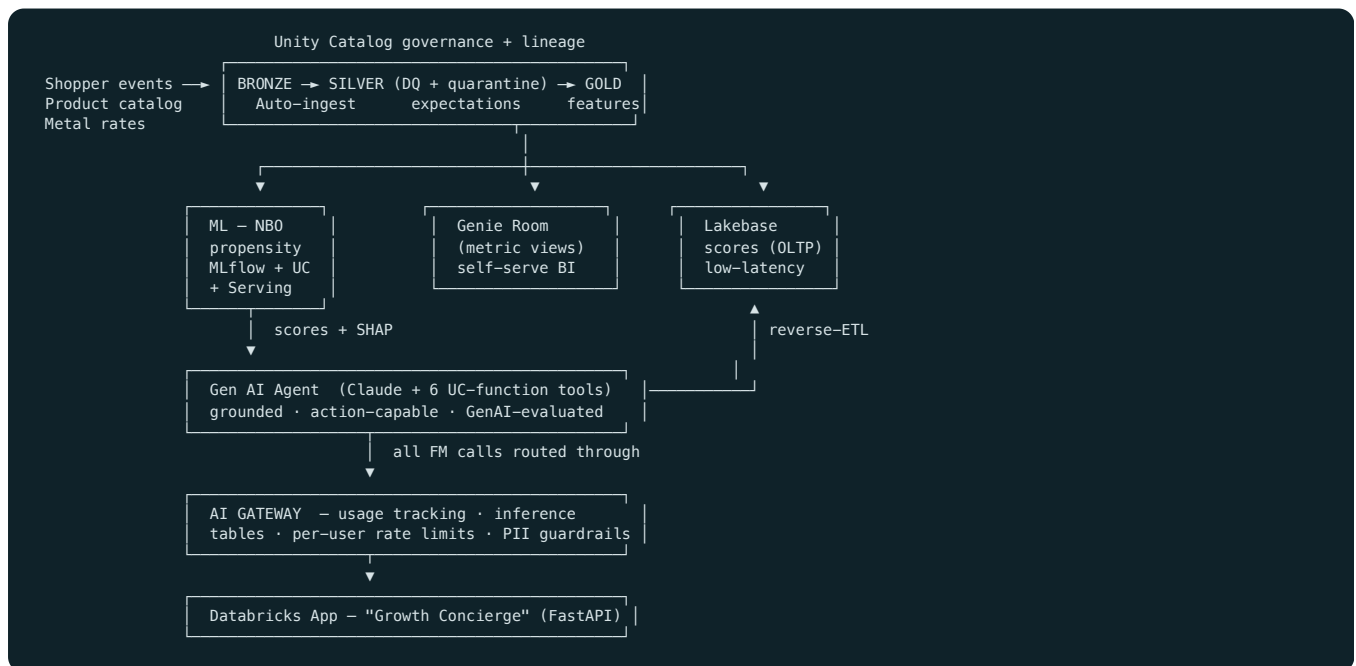
- **Discovery is hard.** Shoppers describe what they want in natural language (“a diamond necklace for an anniversary”), not SKUs. Keyword search misses intent.
- **Pricing is volatile.** Gold and diamond rates move daily; margins erode without live, per-product pricing.
- **Retention beats acquisition.** A repeat buyer is worth far more than a new one — but generic, rule-based nudges (“10% off everything”) waste margin on customers who would have bought anyway and miss the ones who need a reason.
- **The business team is blocked on analysts.** Merchandising and growth owners wait days for a dashboard change to answer a simple question.

The build. A single, connected data journey that turns a rule-based storefront into an **AI-native growth engine**. The centerpiece is a **Next-Best-Offer (NBO) propensity model** that scores every customer’s likelihood to convert on a targeted offer, wrapped in a **Gen AI concierge agent** that recommends the right offer to the right customer — and a **self-serve Genie room** so the business team answers its own questions. Every layer is governed and every foundation-model call is metered and guardrailed.

The golden thread: one seeded behavioral pattern — *high-propensity, diamond-affinity, anniversary-window customers* — flows through every layer, from raw events to the model score to the agent’s action to the app screen.

2. Technical Architecture

The build is **one connected journey, Databricks-native end to end**, in a single Unity Catalog:



Design principles: single catalog and one dataset; the same seeded pattern is traceable at every layer; the ML model *predicts*, the agent *acts*, and every foundation-model call is *governed*; 100% synthetic data with PII masked in Unity Catalog and blocked at the

3. Databricks Components — Deep Dive & Specs

Layer 1 — Lakeflow + Unity Catalog (governed data foundation)

- **Catalog / schemas:** `anuj_vm_workspace_catalog` → `febar_bronze` / `febar_silver` / `febar_gold` / `febar_ml`.
- **Medallion:** Bronze (raw) → Silver (cleaned, typed) → Gold (star schema + `customer_features`, **23,391 customers** with RFM, diamond affinity, engagement, and anniversary-proximity features).
- **Data quality:** Silver **expectations + quarantine** split (`orders_clean` / `orders_quarantine` with `dq_reason`).
- **Governance:** **PII column masks** on customer email & mobile (admins see clear, others masked); **region row filter** on the customer dimension; end-to-end lineage.
- **Lakehouse Monitoring:** snapshot monitor on the model scores table to track drift over reruns.

Layer 2 — Lakebase (operational, low-latency)

- **Instance** `brickjewels-orders` (Postgres 16), **OAuth** credential (no static secret).
- **Reverse-ETL:** Gold model scores synced to `febar_nbo_scores` — **23,391 rows** (High 4,678 / Medium 9,356 / Low 9,357) — for millisecond app reads.

Layer 3 — Machine Learning (deep intelligence)

- **Model:** Next-Best-Offer propensity — **LightGBM**, registered in Unity Catalog as `febar_ml.nbo_propensity` (v1) via **MLflow**.
- **Serving:** real-time **Model Serving** endpoint (`febar-nbo-propensity`) + nightly batch scoring to Gold.
- **Explainability:** **SHAP** — top drivers recovered = *order frequency* › *recency* › *monetary* › *wishlist* › *cart adds* › *diamond affinity*.

Metric	Value	Read
PR-AUC	0.837	vs. 0.455 base rate → 1.84x lift
ROC-AUC	0.854	strong separation
Precision @ top decile	0.969	the top 10% convert ~97% of the time
Brier score	0.155	well-calibrated probabilities

Layer 4 — Gen AI Agent (the connective glue)

- **LLM:** Claude (Databricks Foundation Model API), routed through the AI-Gateway-governed endpoint.
- **6 governed UC-function tools:** `get_customer_propensity`, `get_customer_profile`, `recommend_next_best_offer`, `get_top_prospects`, `search_products`, `get_category_performance` — the model *calls the model* (`get_customer_propensity` invokes Layer 3).
- **Grounded & action-capable:** every fact comes from a tool; no invention.
- **GenAI evaluation** (LLM judge): groundedness **0.8**, correctness **0.8**, safety **1.0** (the single miss was a judge crore-vs-billion unit false-negative — true quality ≈ 5/5).

Layer 5 — Genie (self-serve BI)

- **Metric Views** `mv_sales` and `mv_customer_propensity` for consistent KPI definitions (revenue, units, AOV, diamond mix, avg propensity, LTV) queried with `MEASURE()`.
- **customer_360** view + KPI views + **8 curated seed questions** with join hints and instructions.

AI Gateway — governing every foundation-model call

- Dedicated endpoint `febar-claude-governed` (wraps the shared Claude FM endpoint) — governed **without** touching shared workspace infrastructure.

- **Usage tracking + inference tables** (full request/response logging to Unity Catalog).
- **Per-user rate limit** (60 calls/min) — verified live.
- **PII guardrails (BLOCK)** — verified: a request containing a credit card and email was detected and blocked (`<CREDIT_CARD>` , `<EMAIL_ADDRESS>`). The generic safety filter was deliberately tuned off after it false-positived retail language.

Layer 6 — Databricks App

- **febar-concierge** — build-free **FastAPI** app, deployed and running. Chat with the concierge agent + a customer-360 panel showing propensity, segment, profile, and the SHAP “why”. Integrates every layer above.

4. Further Scope of Improvements

- **Real behavioral signal.** Replace the seeded propensity label with observed outcomes once genuine conversion/redemption events are captured, and retrain on real behavior.
- **Streaming ingestion.** Move Bronze to **Auto Loader / Lakeflow Connect** for near-real-time events and continuous feature freshness.
- **Online feature store & real-time scoring in the loop** so the agent scores brand-new customers on the fly, not just the batch population.
- **Offer experimentation.** A/B test NBO-driven offers vs. control to measure incremental lift and calibrate discount depth to margin.
- **Agent maturity.** Promote the agent to a native **Mosaic AI ChatAgent** with the **Review App**, multi-turn memory, and **Vector Search RAG** over a product + returns/policy corpus for richer grounding.
- **Continuous evaluation & monitoring.** Scheduled GenAI eval (groundedness/safety) + Lakehouse Monitoring alerts on model drift and gateway usage anomalies.
- **Productionization.** Package the whole build as a **Databricks Asset Bundle (DABs)** for one-command, environment-parameterized deployment and CI/CD.

5. Business Value

Outcome	How the build drives it	Signal
Higher conversion	Target offers at the highest-propensity customers instead of blanket discounts	Top decile converts ~97% (0.969 precision)
Margin protection	Personalized offers + live per-product pricing replace blanket markdowns	Discount spend focused where it changes the decision
Repeat purchase / LTV	NBO agent recommends the right offer at the right moment (e.g., anniversary window)	High segment carries ~6x the lifetime value of the Low segment
Analyst self-serve	Genie room over governed metric views	Business answers its own questions; no dashboard queue
Trust & compliance	UC masking + row filters + AI Gateway PII guardrails + full usage logging	Customer PII never leaves governed boundaries
Faster delivery	One reusable, Databricks-native pattern	Portable to any D2C vertical with minimal change

The core shift: BrickJewels moves from *reactive, rule-based merchandising* to a *predictive, governed, AI-native growth loop* — where data quality, an ML model, a Gen AI agent, self-serve BI, and an app are one connected system rather than disconnected tools.

6. Closing

This build demonstrates the full breadth of the Databricks Data Intelligence Platform working as **one integrated journey** — Lakeflow + Unity Catalog, Lakebase, MLflow + Model Serving, the Mosaic AI Foundation Model API with governed UC-function tools, Genie, AI Gateway, and Databricks Apps — all on a single governed dataset, solving a real, specific business problem in D2C fine-jewelry retail.

The ML model genuinely *learns* the pattern (PR-AUC 0.837 vs 0.455 base), the agent genuinely *acts* on grounded data, and every foundation-model call is genuinely *governed*. The result is not a demo of disconnected features — it is a working prototype of how a modern retailer runs growth on Databricks.

Live app: `febar-concierge` · Model: `anuj_vm_workspace_catalog.febar_ml.nbo_propensity` · Governed endpoint: `febar-claude-governed`